POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Enterprise ICT Architectures (Module 1) - Second Project Delivery Neo4j + BPMN

Author(s): **Alberto Lenoci - 11095788**

Elena Casaleggi - 10706547

Matteo Lo Giudice - 10829241

Veronica Fatigati - 11095787

Group Number: **4**

Academic Year: 2024-2025

Contents

1 | Introduction

For the second delivery of this EICTA group project we have recreated the *Well Dressed fashion company* SQL database in non-SQL form.

We have converted the existing database in Neo4j format utilising the Neo4j ETL tool available innately in the Neo4j desktop platform. The tool allows for a direct and precise conversion of every entity, attribute and relation already present in the SQL database, without compromising on database size, accuracy, consistency or cardinalities.

After the creation of the database, we return graph DB data model, briefly describe each entity and the attributes assigned and, lastly, we execute a series of Cypher queries on the database to observe different behaviours and verify efficiency and functionality of the system.

Separate from the DB part of our work, this project also includes the construction a BPMN model for the Well Dressed company, but the BPMN construction will be explored and analysed later in the document in its dedicated section.

2 | Graph DB

The presented diagram showcases a graph-based data model that captures the interconnected structure of entities within the *WellDressed fashion company*. Key entities represented as nodes include *Designer*, *Model*, *Dress*, *Fabric*, *Size*, *Exhibition*, and *Competition*. The interactions among these entities are depicted through relationships such as *DESIGNS*, *WEARS*, *PARTAKES*, *ASSIGNED*, *COMPETES*, *MADE OF*, and *DESCRIBES*.

In graph databases, each node is typically assigned a *Category*, also known as a label, which defines the type or role of the node (e.g., *Designer* or *Dress*). Categories allow for efficient filtering and querying of nodes, enabling targeted operations based on their roles within the graph. This is a key differentiator from traditional ER (Entity-Relationship) models, where entities are defined more rigidly through fixed schemas and attributes.

Unlike the ER Model, where relationships are primarily depicted through foreign keys and require joining tables, graph models leverage direct relationships (edges) between nodes, facilitating more intuitive navigation and exploration of connected data. Graph databases also allow for schema-less design, meaning that properties and relationships can be dynamically added to nodes without requiring changes to a predefined structure, enhancing flexibility and adaptability compared to the more static nature of ER Models.

A graph in this context refers to a data structure consisting of nodes (representing entities) and edges (representing relationships between entities). In non-relational graph databases, data is stored within the graph structure itself, as opposed to being stored in tables like in relational databases.

To better illustrate the graph, the individual nodes representing the various categories and the main relationships have been depicted. As specified in the assignment guidelines, cardinalities were not required and have been omitted for simplicity, allowing for a clearer visualization of the graph.

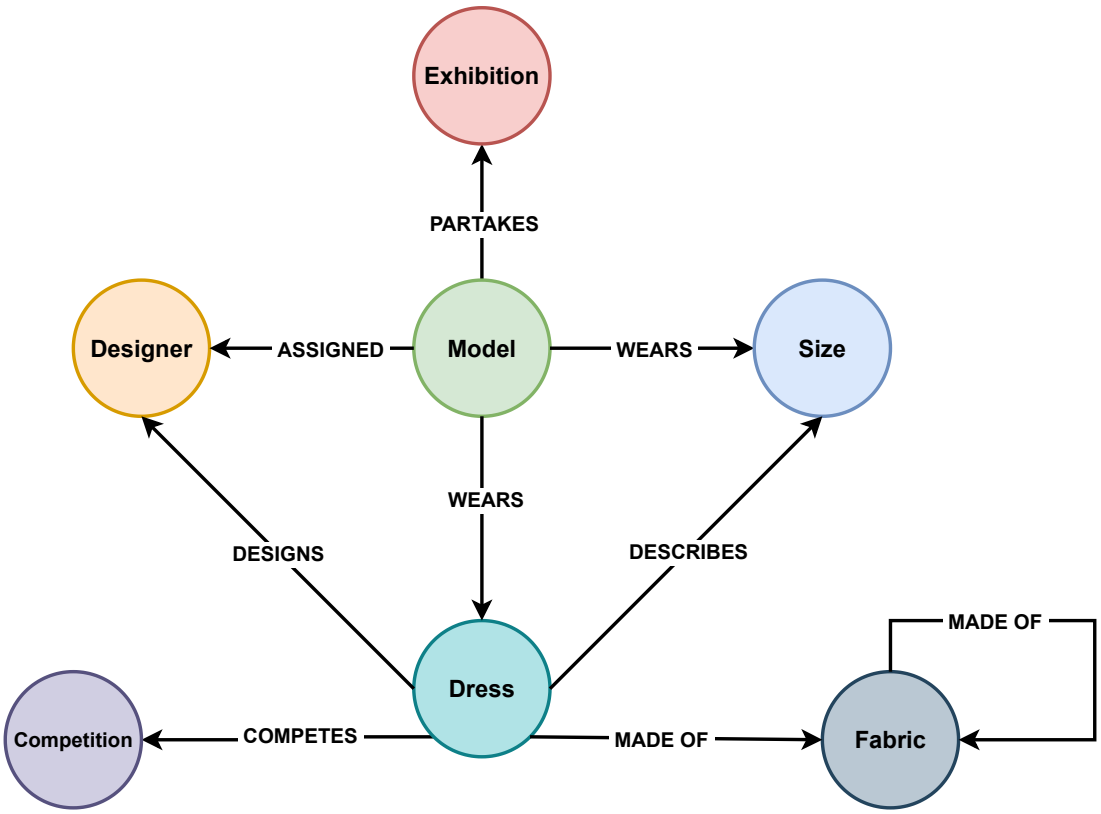

Figure 2.1: Graph DB Data Model

The following cards describe the attributes for each node in the graph-based model of the *WellDressed fashion company*:

Competition

Attributes: `<id>` `cYear`

Dress

Attributes: `<id>` `color` `fabricAmount` `id` `productionTime`

Fabric

Attributes: `<id>` `priceM2` `name` `description` `id`

Designer

Attributes: <id> personalId phoneNumber stageName surname careerDescription name birthDate

Model

Attributes: <id> personalId phoneNumber surname name birthDate height

Size

Attributes: <id> description id

Exhibition

Attributes: <id> duration address description id title

3 | Nodes, Relationships & Attributes

As mentioned in the previous chapter, the relationships present in the model include *Describes*, *Assigned*, *Wears*, *Partakes*, *Made_Of*, *Competes*, and *Designs*. Of particular note is the *Competes* relationship, which includes the attribute *"winner"*. This attribute has been defined as a `Tinyint`, typically representing a binary state indicating whether or not a dress has won a given competition.

The graph-based data model for the *WellDressed fashion company* comprises several nodes, each representing an entity with its associated attributes and their data types. Below is a detailed list of the nodes, their attributes, and a brief description for each:

Table 3.1: Attributes of Node: *Exhibition*

Attribute	**Type**	**Description**
id	Long	Unique identifier for the exhibition.
duration	Long	Duration of the exhibition in minutes.
description	String	A textual description of the exhibition.
address	String	The location address where the exhibition takes place.
title	String	The title or name of the exhibition.

Table 3.2: Attributes of Node: *Size*

Attribute	**Type**	**Description**
id	Long	Unique identifier for the size category.
description	String	A description of the size specification (e.g., XS, S, M, L).

Table 3.3: Attributes of Node: *Model*

Attribute	Type	Description
personalId	Long	Unique identifier for the model.
name	String	First name of the model.
phoneNumber	String	Contact number of the model.
birthDate	String	Birth date of the model, typically represented as a string.
surname	String	Last name of the model.
height	Double	Height of the model.

Table 3.4: Attributes of Node: *Dress*

Attribute	Type	Description
id	Long	Unique identifier for the dress.
color	String	Color description of the dress.
fabricAmount	Long	Quantity of fabric used to produce the dress.
productionTime	Long	Production time required to make the dress.

Table 3.5: Attributes of Node: *Designer*

Attribute	Type	Description
personalId	Long	Unique identifier for the designer.
name	String	First name of the designer.
phoneNumber	String	Contact number of the designer.
birthDate	String	Birth date of the designer, typically represented as a string.
surname	String	Last name of the designer.
stageName	String	Stage name or artistic name used by the designer.
careerDescription	String	Description of the designer's career achievements or focus.

Table 3.6: Attributes of Node: *Fabric*

Attribute	Type	Description
id	Long	Unique identifier for the fabric.
description	String	Description of the fabric.
name	String	Name of the fabric type.
priceM2	Double	Price per square meter of the fabric.

Table 3.7: Attributes of Node: *Competition*

Attribute	Type	Description
cYear	Date	Year of the competition event.

4 | Queries

This chapter contains the results of the utilisation of Neo4j Desktop and Browser platforms to query the database already in our possession from the previous project.

Queries follow the requests of the delivery of the project, with each sub-section of the chapter dedicated to different Cypher statements combinations required.

The queries will be listed below in textual form and Cypher one, including a screenshot of their produced output.

3 Queries with at least 2 nodes in the MATCH statement and conditions

The **MATCH** statement is used to select and find nodes and relations within the database. The following three queries will contain at least two nodes in the **MATCH** statement.

Query 1

Find the first three models wearing size S.

```
1    MATCH (m:Model)-[w:Wears]->(s:Size)
2    WHERE s.description = "Small"
3    RETURN m, s.description
4    LIMIT 3
```

In this query, we search for all Model nodes connected to Size nodes through a *Wears* relation through the **MATCH** statement, then we add a condition through the **WHERE** statement, that of Size being equal to *Small*. We then return, through the **RETURN** statement, the models and the description of the size, limiting to the first 3 entries, as specified in the text of the query.

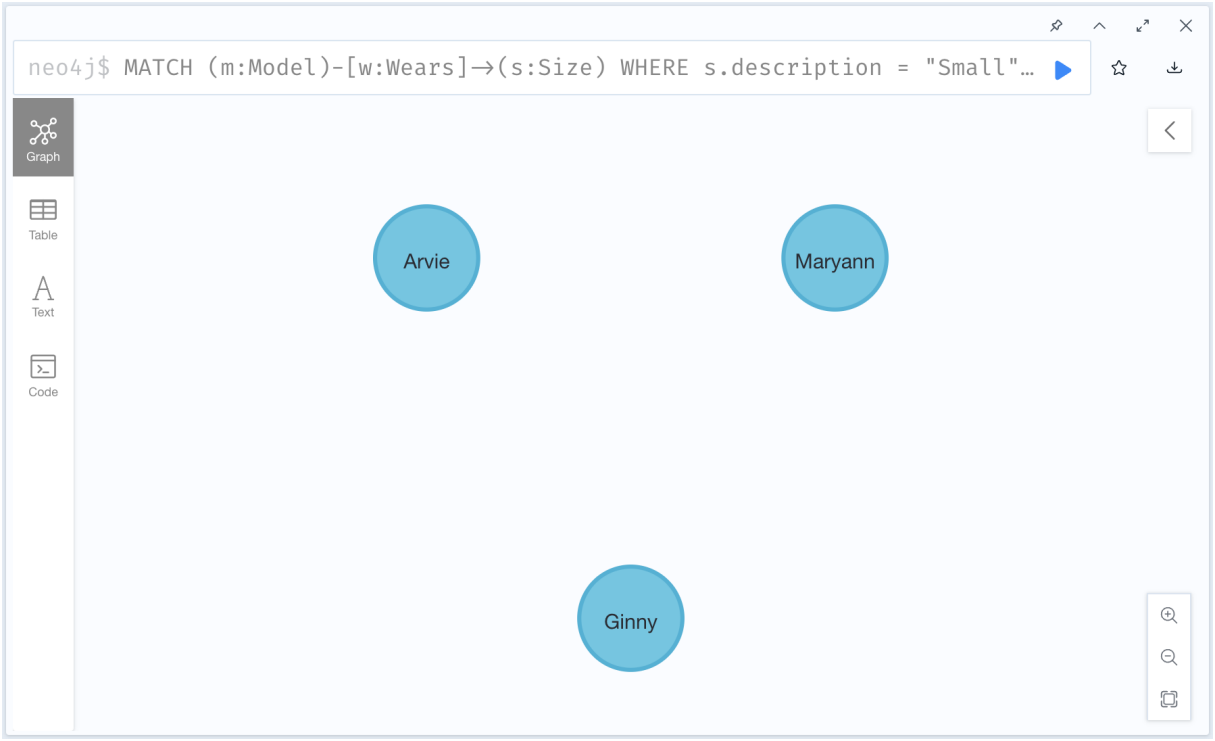

Figure 4.1: Query 1 Neo4j output

Query 2

Find all dresses that have won competitions.

```
1    MATCH (d:Dress)-[r:Competes]->(c:Competition)
2    WHERE r.winner = true
3    RETURN d, type(r), c
```

In the query above, we select all dress nodes and competition nodes connected by a *Competes* relation thorugh the **MATCH** statement, then filter using the **WHERE** one and require for the relationship to have a certain property, the one of being the winner, set as 'true', so only dresses that attended competitions and then won are selected. In the **RETURN** statement, we select as output dresses satisfying the filtered condition, the type of the relation, and the competition.

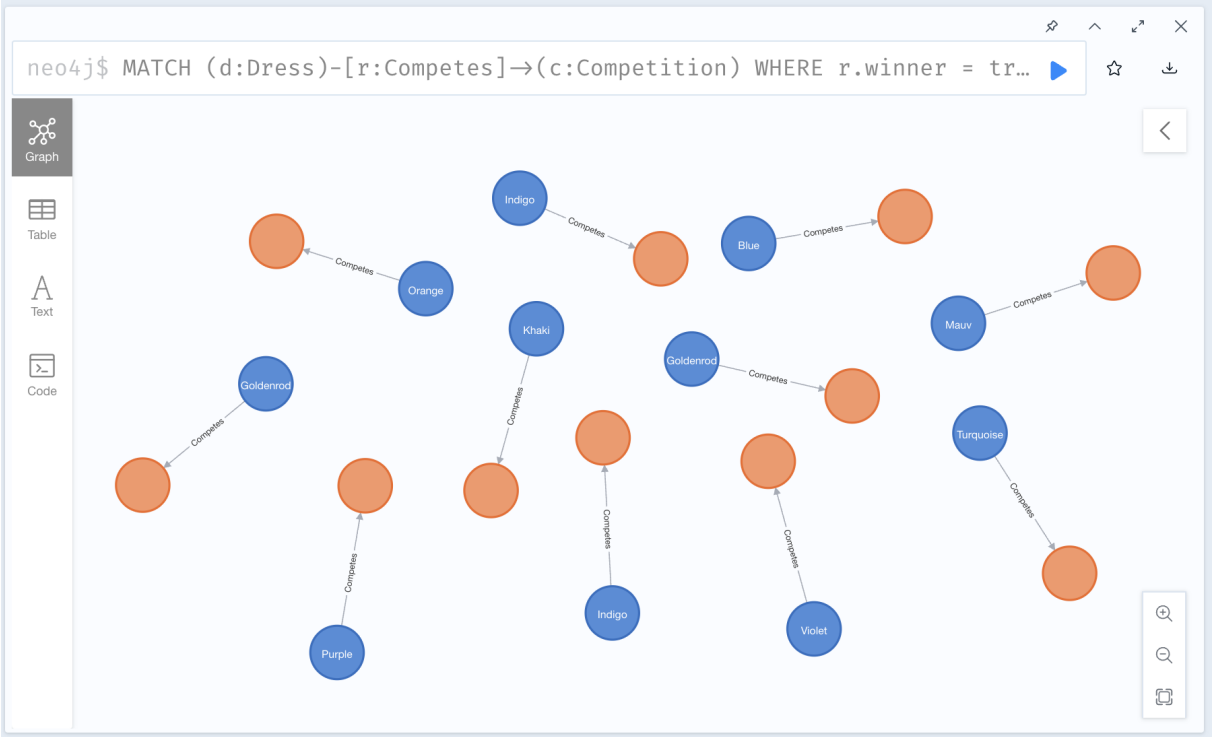

Figure 4.2: Query 2 Neo4j output

Query 3

Find the color of dresses designed by designers whose name starts with letter 'C'.

```
1  MATCH (dd:Designer)-[r:Designs]->(d:Dress)
2  WHERE dd.name STARTS WITH 'C'
3  RETURN DISTINCT dd.name AS Designer_Name, d.color AS Dress_Color
```

This last query for the section again has two nodes in the **MATCH** statement, selecting Designers and Dresses connected by a relation *Designs*. Then through the **WHERE** statement sets the condition of the name of the Designer starting with the letter 'C'. It then returns as output the name of the Designer, and the color of the dress, renaming them both through the **AS** statement.

Figure 4.3: Query 3 Neo4j output

2 queries with at least 2 nodes in the MATCH statement, conditions and aggregation without a WITH statement

Query 1

Find the number of models that took part in all the exhibitions.

```
1    MATCH (m:Model)-[r:Partakes]->(e:Exhibition)
2    RETURN COUNT (DISTINCT m) AS Model_count
```

The query determines the total number of unique models who participated in all exhibitions. It uses the **MATCH** clause to connect Model nodes with Exhibition nodes through the *Partakes* relationship, identifying the models involved in the exhibitions. The **COUNT** function, combined with **DISTINCT**, ensures that only unique models are considered. The query returns the count of such models, giving insights into the overall participation patterns across the exhibitions.

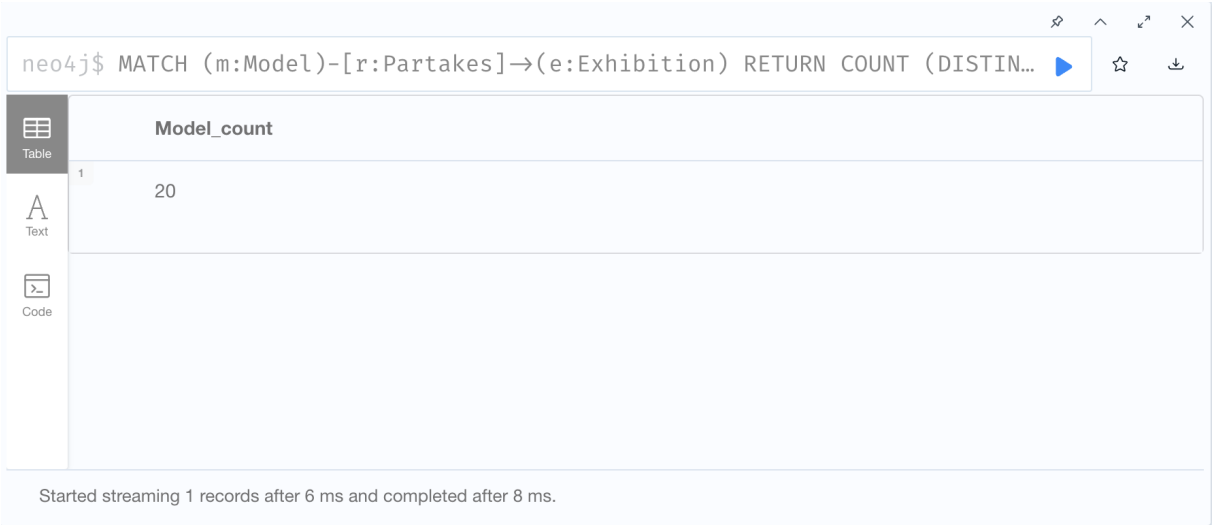

Figure 4.4: Query 1 Neo4j output

Query 2

Find the average fabric amount used to produce dresses in size large.

```
1  MATCH (s:Size)-[r:Describes]->(d:Dress)
2  WHERE s.description = 'Large'
3  RETURN AVG(d.fabricAmount) AS Average_Fabric_Amount
```

This query calculates the average fabric amount used to produce dresses in size large. It uses the **MATCH** clause to link Size nodes to Dress nodes through the *Describes* relationship, identifying dresses of a specific size. The **WHERE** clause filters the results to include only the dresses which size is equal to "Large". Finally, the query applies the **AVG** function to compute the average fabric amount for these dresses and returns the result.

Figure 4.5: Query 2 Neo4j output

2 queries with at least 2 nodes in the MATCH statement, conditions and a WITH statement

Query 1

Find the number of models taller than 180 cm assigned to each designer.

```
MATCH (m:Model)-[:Assigned]->(d:Designer)
WHERE m.height > 180
WITH d, COUNT(m) AS Number_of_Models
RETURN d.stageName AS Stage_Name, Number_of_Models
```

This query calculates the number of models taller than 180 cm assigned to each designer. It uses the **MATCH** clause to connect Model nodes to Designer nodes through the *Assigned* relationship. The **WHERE** clause filters the models to include only those taller than 180 cm. The **WITH** clause groups the data by designer and computes the count of models meeting the height criteria. Finally, the query returns each designer's stage name alongside the count of eligible models.

Figure 4.6: Query 1 Neo4j output

Query 2

Find the average fabric amount used to produce orange, purple, and blue dresses for each fabric.

```
MATCH (d:Dress)-[r:Made_Of]->(f:Fabric)
WHERE d.color IN ['Orange','Purple','Blue']
WITH f, AVG(d.fabricAmount) AS Average_Fabric_Amount
RETURN f.name, Average_Fabric_Amount
```

The query focuses on analyzing the fabric usage patterns by calculating the average amount of fabric consumed for dresses of specific colors. It uses the **MATCH** clause to associate each Dress node with its corresponding Fabric node through the *Made_Of* relationship and applies a **WHERE** condition to filter dresses by color. The **WITH** clause groups the results by fabric type and computes the average fabric amount using the **AVG** function. Finally, the query returns the fabric name along with the calculated average, enabling insights into material usage for different colored dresses.

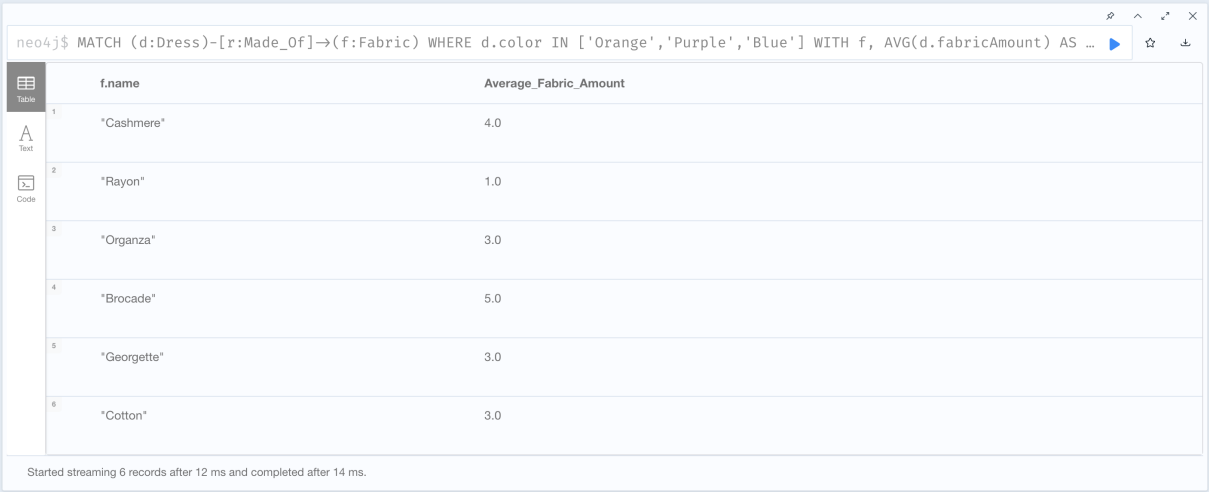

Figure 4.7: Query 2 Neo4j output

2 queries with at least 3 nodes in the MATCH statement, conditions and multiple WITH statements

Query 1

Find the number of 'brocade' (fabric) dresses made by each designer and the number of models assigned to them.

```
MATCH (d:Designer)-[r:Designs]->(dd:Dress)-[rr:Made_Of]->(f:Fabric)
WHERE f.name='Brocade'
WITH d, COUNT(dd) AS Total_Dresses
MATCH (m:Model)-[rrr:Assigned]->(d)
WITH d, Total_Dresses, COUNT(m) AS Total_Models
RETURN d.stageName AS Designer_Name, Total_Dresses, Total_Models
```

This query examines the production focus of each designer on 'Brocade' dresses and the scale of their collaboration with models. By counting both the dresses made from 'Brocade' fabric and the models assigned to each designer, the query provides a clear view of each designer's production level and team size. This comparison can reveal which designers are most active in creating 'Brocade' pieces and may suggest their prominence or specialization within this fabric category.

The query utilizes two **MATCH** clauses to first identify the designers associated with 'Brocade' dresses through a Designs relationship and filters the dresses using a *Made_Of* relationship connected to the Fabric node with the specified name 'Brocade'. The **WITH** clause aggregates the count of such dresses per designer. A second **MATCH** then finds

models assigned to these designers through the *Assigned* relationship, counting them to reflect the scale of collaboration. The **RETURN** statement combines these results, giving a comprehensive view of each designer's activity in terms of dress production and model engagement.

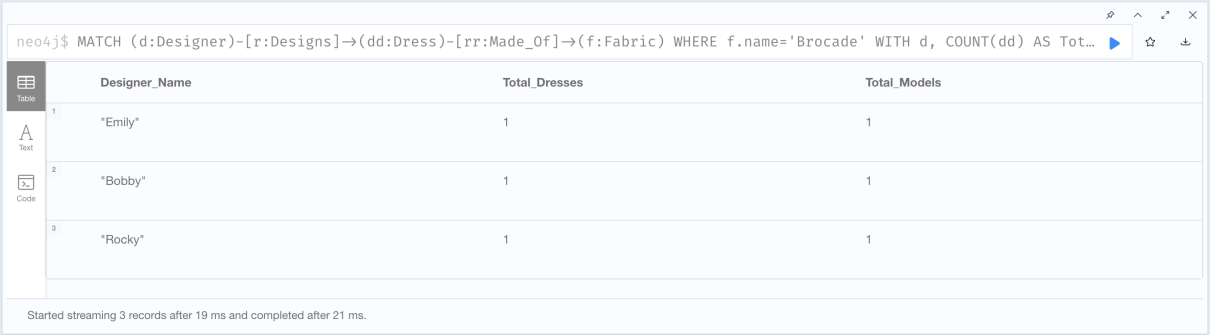

Figure 4.8: Query 1 Neo4j output

Query 2

Find the number of dresses for each size and the number of models shorter than 180 cm who participated in exhibitions for each size.

```
1  MATCH (s:Size)-[r:Describes]->(d:Dress)
2  WITH s, COUNT(d) AS Total_Dresses
3  MATCH (m:Model)-[t:Partakes]->(e:Exhibition), (m)-[y:Wears]->(s)
4  WHERE m.height < 180
5  WITH s, Total_Dresses, COUNT(m) AS Total_Exhibitions
6  RETURN s.description, Total_Dresses, Total_Exhibitions
```

The query begins by matching the relationship *Describes* between Size and Dress nodes, grouping dresses by size and counting them as Total_Dresses. Using **WITH**, it passes this count to the next part of the query.

The second **MATCH** finds models who participated in exhibitions (*Partakes* relationship) and are associated with a particular size of dress through the *Wears* relationship. A **WHERE** clause filters models shorter than 180 cm. The count of such models per size is aggregated as Total_Exhibitions.

Finally, the query returns the size description, total dresses, and the count of eligible models, providing a detailed view of size-based dress distribution and model participation.

Figure 4.9: Query 2 Neo4j output

1 query with the shortestPath(...) function

Query 1

Find the shortest path between the designer 'Wilson' and the designer 'Iris'.

```
1  MATCH p= SHORTESTPATH((d:Designer {stageName: 'Wilson'})-[*]-(dd:
   Designer {stageName: 'Iris'}))
2  RETURN p
```

This query utilizes the **SHORTESTPATH** function in Cypher to determine the minimal traversal path between two designer nodes labeled 'Wilson' and 'Iris'. The **MATCH** clause specifies a variable **p** representing the shortest path connecting the nodes `(d:Designer {stageName: 'Wilson'})` and `(dd:Designer {stageName: 'Iris'})`. The **SHORTESTPATH** function restricts the traversal to find the minimal number of hops or connections required between these nodes. The `[*]` symbol within the relationship pattern indicates that any number of intermediate relationships may exist along the path, either directed or undirected. By employing this function, the query optimally computes and returns the path **p**, allowing for efficient exploration and analysis of the shortest connection in the graph.

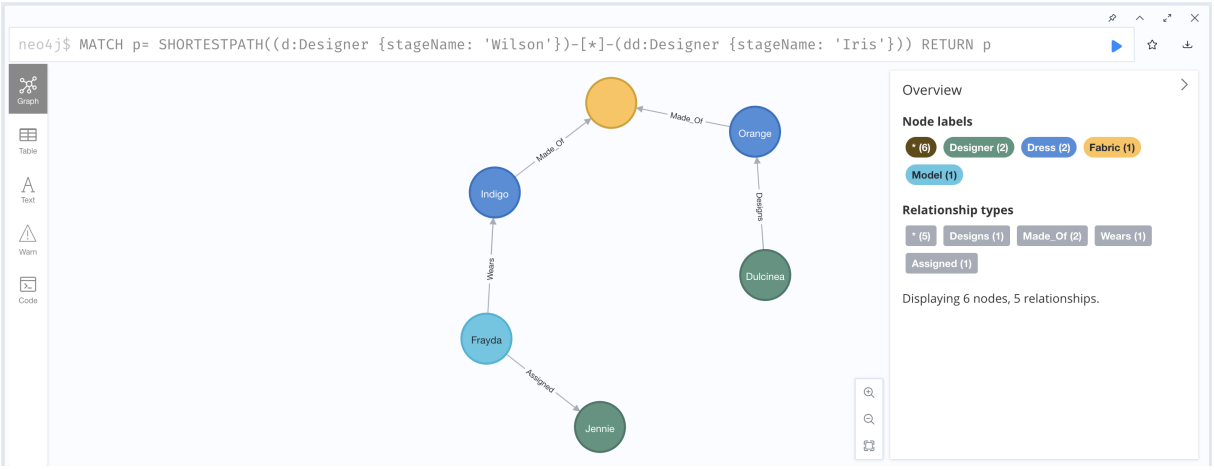

Figure 4.10: Query 1 Neo4j output

5 creation/update/delete queries

Query 1

Create a new exhibition with random attributes and the necessary relations with a random model. Random attributes and model can be chosen utilising mockaroo like in the first project.

```
1   MATCH (m:Model {personalId:12})
2   CREATE (e:Exhibition {duration: 120, address: '4 Georgetown Road',
    description: 'Pre-season Friendly', id: 26, title:'Naive Chic'})
3   CREATE (m)-[:Partakes]->(e)
4   RETURN m, e
```

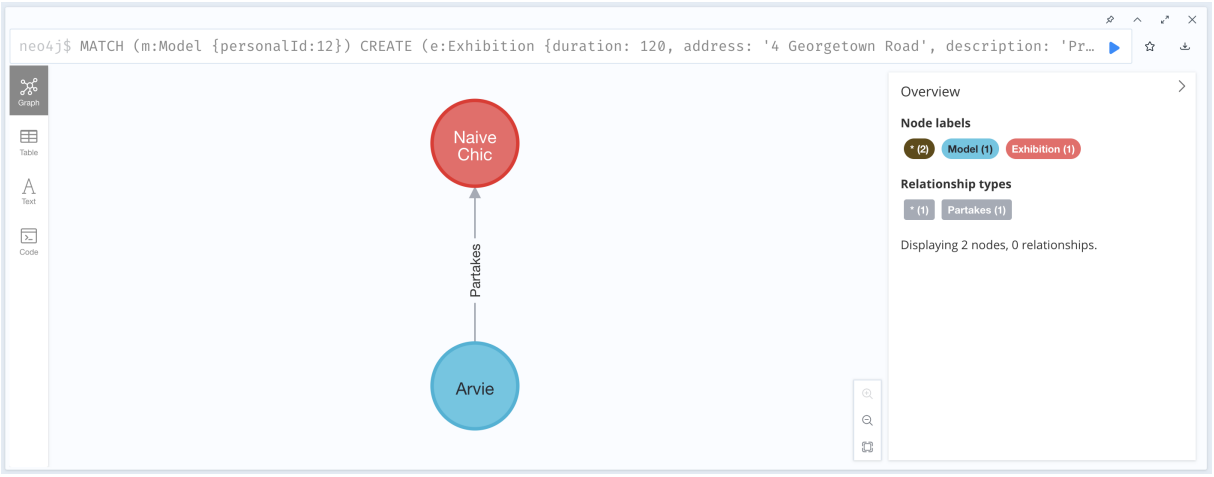

Figure 4.11: Query 1 Neo4j output

Using mockaroo, we created a new exibition with new attributes. These attributes, given the SQL nature of the data generation, include the personal id of the model to assign. We match the individual model and assign it an alias. We then create a new exhibition with the given attributes and create a relationship between the chosen model and the newly created exhibition.

Query 2

Update the size of a randomly chosen Model from S to M.

```
MATCH (m:Model {name: 'Cris'})-[r:Wears]->(s:Size)
SET s.description = 'Medium'
RETURN *
```

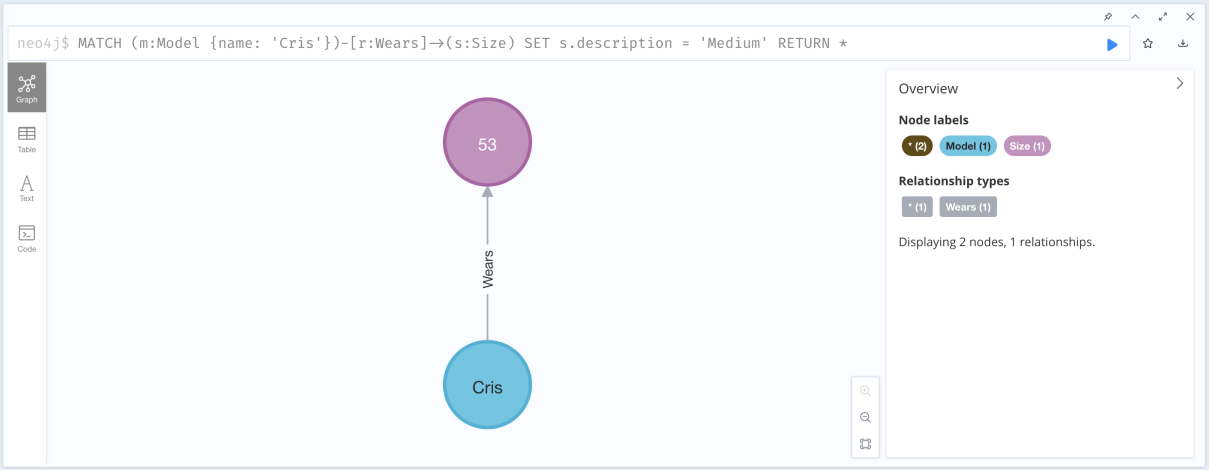

Figure 4.12: Query 2 Neo4j output

Choosing a random S-sized model, we run a simple query to update its relationship with the "Size" nodes.

Query 3

Delete all the models whose height is < 180cm.

```
MATCH (m:Model)
WHERE m.height < 180
DETACH DELETE m
```

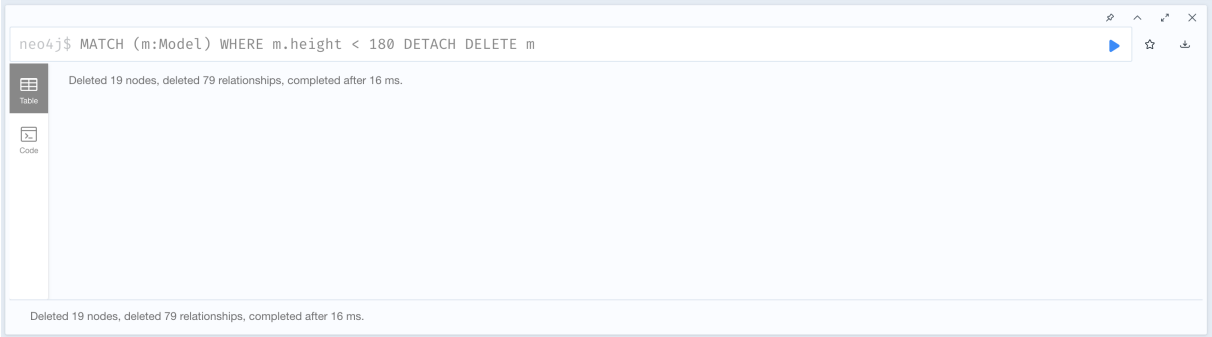

Figure 4.13: Query 3 Neo4j output

The double data type used in the height attribute allows us to easily filter out models under 180cm, leading to a quick deletion. First of all, we used the command **DETACH** in order to eliminate the relationships among nodes, and then the **DELETE** command to cancel the node itself.

Query 4

Create a winning relationship between competitions and the designers whose dresses have won said competitions.

```
1  MATCH (d:Designer)-[r:Designs]->(dd:Dress)-[rr:Competes]->(c:
   Competition)
2  WHERE rr.winner = true
3  CREATE (d)-[:winner]->(c)
4  RETURN *
```

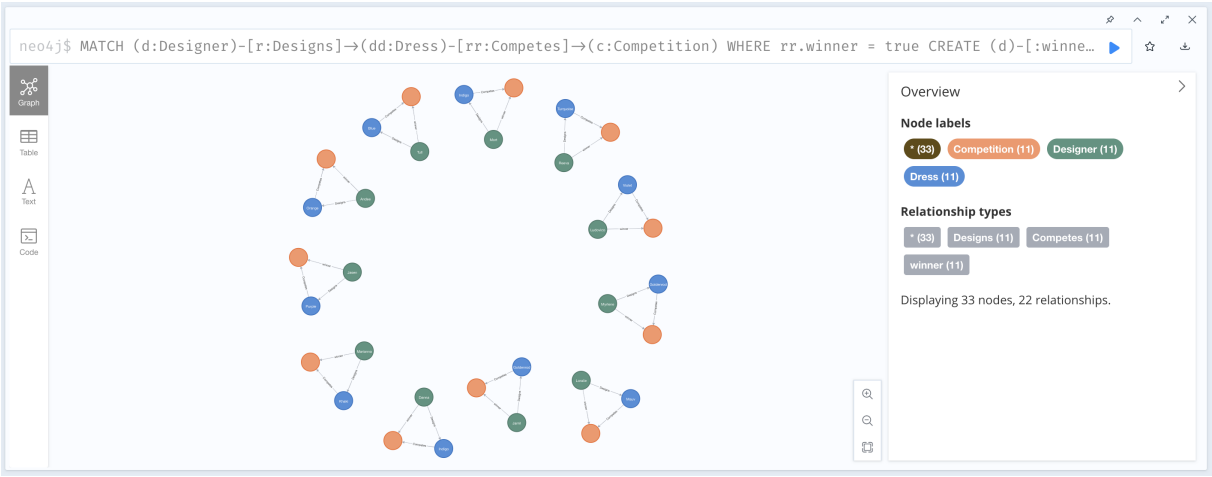

Figure 4.14: Query 4 Neo4j output

Since it is possible to follow a straight path between Designers and Competitions, we

optimize the query by only using 1 complex pattern in the **MATCH** statement instead of 2 simple ones. We then filter for the winning relationships and create new relationships using the aliases.

Query 5

Update the prices of fabrics due to a 8% inflation rate.

```
1    MATCH (f:Fabric)
2    SET f.priceM2 = f.priceM2 * 1.08
3    RETURN f.name AS Fabric_Name, f.priceM2 AS New_Price
```

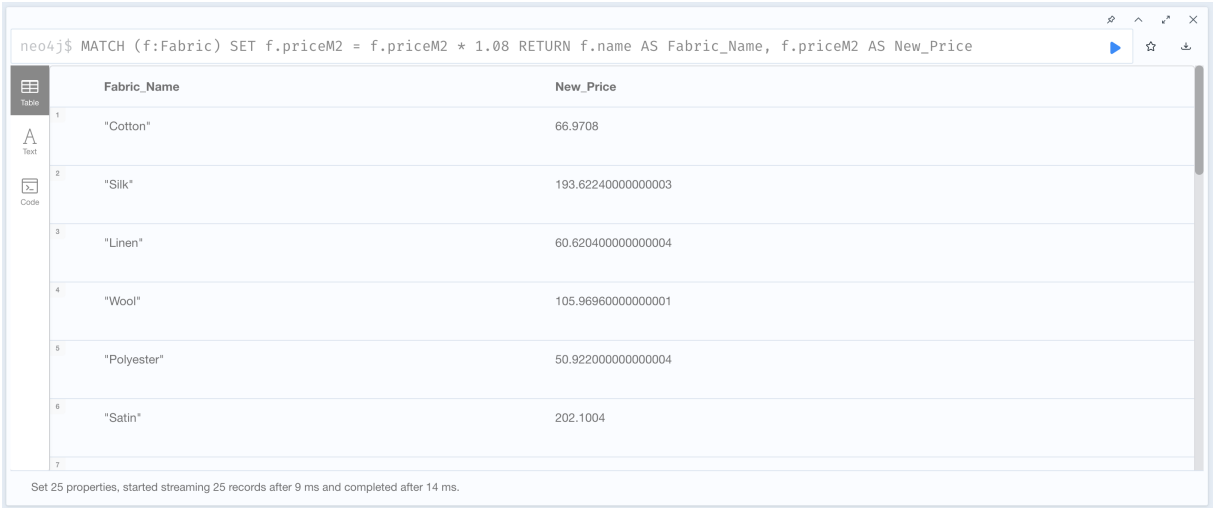

Figure 4.15: Query 5 Neo4j browser output

As the double data type allows for arithmetic computation, updating the values is a simple matter. Macroeconomics hit the *WellDressed fashion company* hard.

5 | BPMN

In this section we build a **BPMN** model for the *WellDressed fashion company* based on the delivery here reported:

> "Given the following process, write the BPMN diagram representing it. Models submit their CVs through the company's front desk. As soon as a CV is received, it is forwarded to the HR department, which checks it. If they deem the candidate suitable for a position, they forward the CV to the head of the company. Otherwise, the candidate is noticed, and the process terminates. The head of thecompany contacts the fashion designers (internals) who work for the company and submits the model's CV to them. Designers have three days to reply to the head (a response is mandatory, even if negative). If a designer is interested in working with the model, the head asks the front desk to contact the model to set up a meeting with them. If no designer is interested, the candidate is notified, and the process terminates. If the model answers affirmatively, the designer is contacted through the front desk, and they are asked to prepare a dress for the model. The dress is sent to the model, and then a meeting with the head is held. After the meeting, the head of the company provides their final decision through the front desk. They have at most 5 days to do so. Otherwise, the candidate is notified, and the process terminates."

While most of the reported process can be directly converted to BPMN format without trouble, some points require us to make particular assumptions, here listed:

- From the text we know that "if no designer is interested, the candidate is notified, and the process terminates". Since it wasn't specified how the candidate is notified, we assumed that the Head of the company forwarded the rejection to the HR department, who is then in charge of notifying the candidate. This assumption aligns with the process where HR manages candidate rejections based on CV evaluations.
- A parallel multi-instance task was used for the designers evaluating the CVs because the text specifies that multiple designers receive the CV simultaneously and must

independently provide their responses within three days. This BPMN construct allows for modeling tasks that occur concurrently across several participants, ensuring that each designer evaluates the CV separately but within the same timeframe. Using this approach accurately represents the simultaneous and independent nature of the designers' evaluations while maintaining clarity and alignment with BPMN standards for parallel processes.

- In addition, a separate pool was created for the meeting involving the Head of the Company, Designer, and Model because the Model, being external to the Well-Dressed Fashion Company, exists in a different organizational context. BPMN conventions require distinct pools to represent entities that operate independently, so the Model cannot directly interact within the company's internal pool. By creating a shared pool for the meeting, the diagram ensures that these interactions, which bring together internal (Head and Designer) and external (Model) participants, are accurately and clearly represented as occurring in the same space at the same time, enabling a cohesive depiction of the collaborative process.
- Finally, no distinction between a positive or negative outcome for the final decision was made in the BPMN diagram because the text does not specify any differences in the process depending on the result. Both outcomes, acceptance or rejection, are described uniformly, with the Head of the Company communicating the decision through the front desk within the given five-day timeframe. Without explicit details on how the processes for positive and negative decisions diverge, the BPMN reflects a single, unified flow to represent the decision communication, ensuring consistency with the provided information.

We have then used the *demo.bpmn.io* platform to design the BPMN model reported in the following page.

To download the BPMN model, please **click here**.

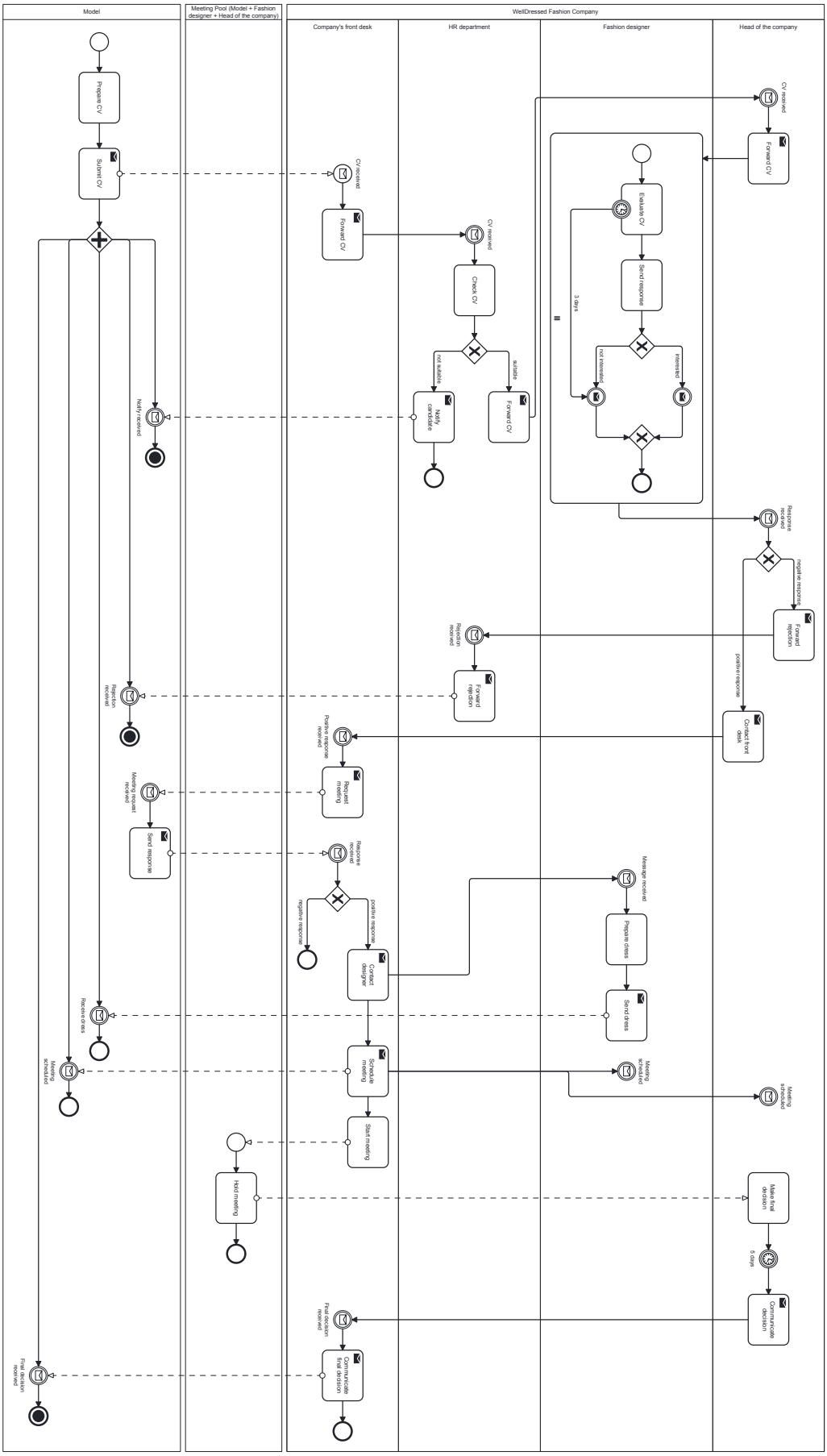

Figure 5.1: BPMN

List of Figures

List of Tables